

Design Report

Safe Semantic Planner in a Finite Cartesian State Space

1. Introduction

This project implements a safe planner for a finite Cartesian state space. The planner receives an initial state, a goal state, bad states, and directed transitions. Each transition has a cost, safety score, reliability, and availability flag.

The planner must reach the goal without visiting bad states while considering path cost, safety, and reliability.

2. Problem Definition

A finite state set is represented as:

$$S = \{s_1, s_2, \dots, s_n\}$$

Each state has a Cartesian embedding:

$$s_i = (x_1, x_2, \dots, x_d)$$

The planner receives:

- Initial state
- Goal state
- Bad states
- Directed transitions

3. Objectives

The implementation aims to:

1. Reach the goal.
2. Never visit a bad state.
3. Minimize total transition cost.
4. Maintain a large minimum Euclidean distance from bad states.
5. Produce a solution within reasonable execution time.
6. Support replanning after environmental changes.

4. State Representation

Each state contains:

- A unique 64-bit ID
- A vector of Cartesian coordinates

Example:

$$S = (0, 0)$$

$$A = (1, 0)$$

$$G = (3, 0)$$

5. Transition Representation

Each directed transition contains:

- Transition ID
- Source state
- Destination state
- Cost
- Safety
- Reliability
- Availability

6. Data Structures

The implementation uses:

- `unordered_map` for state lookup
- Adjacency lists for outgoing transitions
- Reverse adjacency lists for incoming transitions
- `unordered_set` for bad states
- `unordered_map` for `g` and `rhs` values
- Priority queue for the OPEN set

These structures allow efficient storage and lookup of states, transitions, bad states, and search values.

7. Algorithm

The planner uses a D* Lite-style incremental search structure.

The important values are:

- $g(s)$: current best known cost-to-go.
- $rhs(s)$: one-step lookahead value.

For a non-goal state:

$$rhs(u) = \min(c(u, v) + g(v))$$

over valid outgoing transitions.

The goal is initialized with:

$$rhs(goal) = 0$$

The priority key is:

$$K(s) = [\min(g(s), rhs(s)) + h(s) + km, \min(g(s), rhs(s))]$$

A priority queue is used to process states according to their calculated keys.

8. Heuristic

The heuristic is the Euclidean distance from the current state to the goal:

$$h(s) = \text{distance}(s, \text{goal})$$

For Cartesian vectors:

$$\text{distance}(a,b) = \sqrt{\sum((a_i - b_i)^2)}$$

This heuristic guides the search toward the goal using the geometric positions of the states.

9. Safety Computation

For every state, the minimum distance to a bad state is calculated:

$$D(s) = \min \text{distance}(s, b)$$

where b belongs to the set of bad states.

A larger value indicates that the state is farther from the nearest bad state.

Bad destination states are completely forbidden by the planner.

10. Edge Cost

The implementation combines:

- Base transition cost
- Transition safety penalty
- State safety penalty
- Reliability penalty

Conceptually:

$$\text{combined cost} = \text{transition cost} + \text{safety penalties} + \text{reliability penalty}$$

The edge-cost calculation therefore allows the planner to consider not only the base cost of a transition but also its safety and reliability.

11. Dynamic Replanning

The implementation supports:

- Transition availability changes
- Transition addition
- Transition removal by disabling
- Goal updates
- Bad-state updates

After a change, the graph is updated and the planner recomputes the search on the changed state space.

The implementation retains the D* Lite-style g , rhs , and priority-queue structures while using fresh initialization for robust handling of the assignment's dynamic test cases.

12. Test Cases

Test Case 1: Basic Reachability

Graph:

S -> A -> B -> G

Expected result: a valid path from the initial state to the goal is returned.

Actual result:

0 -> 1 -> 2 -> 3

Test Case 2: Bad State Avoidance

The graph contains a potentially dangerous route:

S -> A -> X -> G

where X is a bad state.

The planner must not visit state X.

Actual result:

0 -> 1 -> 2 -> 3

State 6, representing X, is not visited.

Test Case 3: Safety Margin

State 2 is marked as a bad state. The planner therefore avoids the route through state 2 and selects an alternative safe route.

Actual result:

0 -> 4 -> 5 -> 3

Test Case 4: Dynamic Transition

Initially, the planner selects:

0 -> 1 -> 2 -> 3

Transition 2 is then made unavailable.

The planner replans and selects:

0 -> 4 -> 5 -> 3

This demonstrates dynamic replanning after a transition becomes unavailable.

Test Case 5: Goal Update

The original goal is state 3.

The goal is then changed to state 5.

The planner produces the updated route:

0 -> 4 -> 5

This demonstrates goal-update handling.

Test Case 6: Transition Addition

A new shortcut transition is added to the graph.

Before the shortcut, the planner selects:

0 -> 1 -> 2 -> 3

After adding the shortcut, the planner finds:

0 -> 9 -> 3

The path cost decreases from 6.0000 to 3.5000.

13. Experimental Results

The following results are taken from the actual execution output stored in `results/results.txt`.

Test	Success	Bad States Visited	Path	Path Cost	Minimum Safety Distance	Reliability	Explored States
TC1	Yes	No	0 → 1 → 2 → 3	6.0000	1.0000	0.8574	4
TC2	Yes	No	0 → 1 → 2 → 3	6.0000	1.0000	0.8574	9
TC3	Yes	No	0 → 4 → 5 → 3	7.5000	1.0000	0.7290	8
TC4	Yes	No	0 → 4 → 5 → 3	7.5000	1.4142	0.7290	8
TC5	Yes	No	0 → 4 → 5	5.0000	2.0000	0.8100	3
TC6	Yes	No	0 → 9 → 3	3.5000	1.4142	0.9405	8

The program also reports planning time and approximate memory usage.

For the small demonstration graph, the displayed planning time is 0.0000 ms because the execution completes faster than the displayed precision.

The approximate memory values range from 1696 to 1808 bytes for the major containers used by the demonstration.

14. Time Complexity

Let:

- V = number of states
- E = number of directed transitions

Graph construction is approximately:

$$O(V + E)$$

The search uses a priority queue and processes graph states according to their keys. The practical amount of work depends on the number of states and transitions processed and on the changes introduced during replanning.

For the implementation, the exact runtime also depends on the number of affected states, available transitions, and priority-queue operations.

Therefore, the practical complexity of replanning depends on the changed portion of the graph rather than only on the total graph size.

15. Space Complexity

The major structures store:

- States
- Transitions
- Outgoing adjacency lists
- Incoming adjacency lists
- g values
- rhs values
- Bad-state set
- OPEN priority queue

The overall storage requirement is approximately:

$$O(V + E)$$

apart from container and priority-queue overhead.

16. Advantages

- Avoids explicitly marked bad states.
- Uses Cartesian geometry.
- Considers transition cost, safety, and reliability.
- Supports dynamic changes.
- Supports goal updates.
- Produces measurable experimental results.

- Provides both normal and dynamic test cases.

17. Limitations

- The demonstration graph is small and synthetic.
- The heuristic is geometric and does not learn from previous searches.
- Safety and reliability weights are manually selected.
- The printed memory figure is an approximate accounting of major containers.
- The dynamic replanning implementation recomputes the search state after changes rather than providing a fully optimized incremental update of every affected state.

18. Conclusion

The project demonstrates a safe semantic planner for a finite Cartesian state space. It combines graph search, heuristic guidance, safety computation, transition reliability, and dynamic replanning.

The six test cases demonstrate basic reachability, bad-state avoidance, safety handling, dynamic transition changes, goal updates, and transition additions.

The experimental results show that the planner can find valid paths while avoiding explicitly marked bad states and can adapt when the planning environment changes.